

Generative Model-Driven Code Creation and Vulnerability Analysis for Developmental Applications

Noah Reneau, Kyle Stein, Brian Walker, Seth Vinson, Lanier Watkins
Department of Computer Science, Johns Hopkins University, Baltimore, USA

Abstract—With the recent societal implications of generative artificial intelligence (AI), particularly in the realm of software development, concerns about the reliability and security of AI-generated code becomes paramount in the minds of researchers and organizations alike. This paper examines whether large language models (LLMs) can develop effective and secure tools for auditing AI-developed autonomous agents in a competitive game environment. To model this scenario, we focus on a modified version of weighted Rock-Paper-Scissors, specifically Glico, as a testbed for adversarial AI competition. We then prompted the LLM to generate various strategy bots to play the game, including: a pattern-based bot, a cyclic-strategy bot, a random-move bot, and a reinforcement learning bot using Q-learning. The bot development process was iterative, with the LLM refining code in response to functional errors identified through custom dynamic testing strategies. We evaluated the performance of each bot (win/loss rates, time to win), as well as analyzed the security of the AI-generated code using static analysis tools. Our results demonstrate that while LLMs can generate functional game-playing agents, doing so requires multiple prompt iterations and human guidance. Even then, security vulnerabilities remain when deploying AI-generated code in user-created environments like the tested gaming platform. The results of this project aim to provide insight into the practical potential of LLMs to develop AI within restricted, goal-driven environments.

Index Terms—Static Code Analysis, Software Security, Artificial Intelligence, Threat Modeling, Secure Software Engineering, Machine Learning

I. INTRODUCTION

The rapid evolution of AI has opened up numerous growth opportunities in the realm of automating various aspects of software engineering [1]–[3]. Generative AI models, such as OpenAI’s ChatGPT [4] or Meta’s Llama model [5], have demonstrated impressive capabilities when it comes to producing functional code, offering potential to augment or even replace certain aspects of the software development process. However, a major blockade towards the integration of AI into critical workflows lies in significant concerns related to the effectiveness and security of the AI-generated code, as well as its ability to audit its own output to ensure safety and resilience against vulnerabilities.

These questions are particularly timed in the context of modern software development, where increasing code complexity and the pace of innovation challenge human developers to maintain high-quality, secure code. By investigating LLM-generated code in a constrained, competitive game environment, we explore the potential of AI to serve as both a developer and an auditor. If these foundation models can autonomously generate effective, secure code, it could signif-

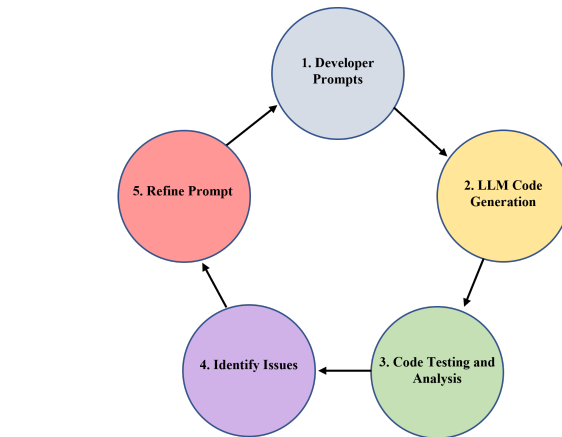


Fig. 1. Code Development Cycle Diagram: (1) specify game rules and desired behavior; (2) generate Python bot code via ChatGPT; (3) test and analyze code in the game environment; (4) identify bugs and inefficiencies; (5) refine the prompt and repeat.

icantly reduce the development burden on human engineers while simultaneously enhancing productivity and address the growing shortage of cybersecurity expertise. Such capabilities might pave the way for the automation of entry-level software development tasks, allowing human engineers to focus on higher-level problem solving and innovation.

Specifically, we ask: *Can AI reliably build secure, autonomous systems and audit its own code?* To answer this, we task ChatGPT with designing and evaluating agents in a custom cyber-gaming setting inspired by DARPA’s Cyber Grand Challenge [6]. Our domain is a modified version of Glico [7], extending weighted Rock-Paper-Scissors to Rock-Paper-Scissors-Lizard-Spock with syllable-weighted scoring to introduce strategic depth in a simple game environment. In order to add another layer of complexity to our Rock-Paper-Scissors-Lizard-Spock (RPSLS) game, we took the weighted RPS concept from Glico and added point values to the winning choice depending on how many syllables the winning choice is. So for instance, if the winning choice is rock, the bot that chose rock will get one point. This turns the game into a less random game and more into a game of strategy. This was done in order to provide a simple yet non-trivial domain for AI bot complexity. We generate multiple bot agents, some AI-driven others pattern-based, and evaluate their performance through dynamic Monte Carlo simulations. Finally, we subject the AI-generated code to static security analysis using GitLab’s SAST [8] and the Bandit Python

library [9] to identify vulnerabilities and assess auditability.

Our main contributions consist of developing a gaming environment as a testbed in order to assess the effectiveness and security concerns of LLM generated code and its place in the development workflow. This was accomplished through the creation of four distinct bots playing against each other within the pre-defined environment. These contributions aim to showcase the current limitations and considerations of generative models when placed into existing developer workflows.

II. RELATED WORK

Existing work in the realm of LLMs for developing secure agents can be summarized by key efforts in AI code generation. Kholoosi et al. [10] performed an empirical study to analyze the thoughts of researchers that used LLMs for secure software engineering. They determined that LLMs are adept at vulnerability detection, information retrieval, and penetration testing. The researchers built experiments around these findings using the LLM as an Oracle to test the feasibility of using this approach in real-world settings. They determined that this approach most contributed generic security information, and thus was probably not feasible for industry use. Our work differs in that we leverage LLMs directly to build AI agents and performed static analysis on the code.

Hossain et al. [11] acknowledge that LLMs can be used for secure code development and attempt to determine the types of security threats that could compromise it. They use the STRIDE threat modeling framework to develop mitigations for common security threats that may attempt to exploit LLMs when used to develop secure code. However, our work does not analyze LLMs for security vulnerabilities; instead, we focus on the development of AI agents, and perform static code analysis looking for vulnerabilities in the AI agent code developed by the LLM.

K. Raghi et al. [12] utilized LLMs to automate aspects of the software development lifecycle (SDLC) with the goal of streamlining the entire process. The authors gathered datasets to train LLMs to perform project planning, code generation, and testing. The authors evaluated their approach by leveraging the LLMs to automatically develop several sample applications. They also developed these applications manually to compare and contrast the different approaches. Finally, they performed static code analysis on the code generated by the LLMs. However, our work is unique since we use an LLM to develop fully functional AI agents that demonstrated their functionality through their ability to strongly compete against each other, whereas the authors in this study simply developed sample applications.

III. METHODOLOGY

We adopted an iterative prompt-based methodology to generate the AI bots' code using OpenAI's ChatGPT which is visualized in Figure 1. In each iteration, we first provided the model with a detailed specification of the game rules,

TABLE I
RPSLS WIN CONDITION CHART

Choice	Beats	Points
Rock	[Scissors, Lizard]	1
Paper	[Rock, Spock]	2
Scissors	[Paper, Lizard]	2
Lizard	[Spock, Paper]	2
Spock	[Scissors, Rock]	1

win condition chart, referenced in Table I, objectives, and desired bot behavior with the notable exception of secure code practices, shown as Step 1. The model was then tasked with producing Python code for a game-playing bot shown as Step 2. After obtaining an initial code draft, we tested the code for functionality with a pre-developed gaming environment. This entailed running the code within the environment and looking for unwanted behavior such as infinite loops, lack of exit control, and general code quality metrics. This composes Step 3 in the figure. Any issues identified (bugs and inefficiencies, Step 4) were fed back into subsequent prompts for refinement, shown as Step 5. For example, if the bot's logic had corner-case bugs or suboptimal error handling, we prompted ChatGPT with these findings and requested improvements. We sometimes asked ChatGPT to review its own output for problems as well, leveraging a self-refinement style of prompting, as detailed in Tony et al [13]. Through this iterative cycle the bots' code was progressively improved until it met functional requirements. The final outcome of this process were four robust bots, all produced via ChatGPT and ready for integration into the game environment.

A. Cyber Gaming Environment Design

To prepare the gaming environment, we designed a self-contained PyGame [14] simulation to host automated RPSLS matches. The environment was created with inspiration from CGC-like competitions, meaning that it was built to support fully autonomous, repeatable battles between bots with no human intervention needed once a match starts. The resulting implementation defines the game rules, orchestrates turn-taking, and records outcomes of matches. To illustrate the environments core logic, Figure 2 shows a simplified excerpt of the match running loop.

```

max_rounds = 1000
round = 0
while rounds < max_rounds:
    moveA = botA.choose_move(botB_history)
    moveB = botB.choose_move(botA_history)
    if moveA == moveB:
        rounds += 1
        continue
    winner = determine_winner(moveA, moveB)
    return winner, rounds+1
return None, max_rounds

```

Fig. 2. Simplified Environment Game Logic

The environment was designed independently of the bots in order to more accurately depict the way in which AI generated code is used, namely in conjunction with existing, human-driven development artifacts. Key features of the environment include logic for the match structure, timeout logic for stalemates, and logic pertaining the interface and extensibility of the environment.

1) *Match Structure*: Rather than a single throw deciding a winner, we implemented the concept of a match which consists of repeated rounds until one bot hits the point threshold of 100. In each round, both bots choose a move simultaneously and the outcome is win/lose for one side or a tie if the moves are the same.

2) *Timeout for Stalemates*: To prevent cases of endless ties or repeating loops, specifically in our dynamic testing, we also set an upper limit on rounds per match of 1000 rounds. If this limit is reached without either bot hitting the target score, the match is declared a draw by timeout. This ensures that the Monte Carlo dynamic testing can progress without hanging on an infinite game

3) *Interface and Extensibility*: The environment was coded in such a way as to allow new bots to be easily added as needed. Each bot is expected to have a method called *choose_move(history)* that returns one of the five moves where history is a record of the last move played by the opponent bot to enable our reinforcement and statistical analysis bots to play with the previous move accounted for. The environment calls both bots' move functions each round, compares the moves using the win chart, and keeps track of the number of rounds. It then returns a winner and statistics such as rounds played and percentage of games won.

IV. RESULTS, EVALUATION, AND DISCUSSION

A. Bot Development with ChatGPT

Our experimentation with bot development using ChatGPT (GPT-4o-mini) yielded four distinct game-playing bots (PatternBot, RandomBot, QLearningBot, CycleBot) that were utilized in the experiment. Each agent was developed by first prompting the LLM with a description of the task and constraints, then refining the generated code through testing until a stable, functional version was reached. Bot and environment development was done in Python 3 on a Windows 11 PC (Intel i7-9700K, 16 GB RAM, 932 GB storage). Once finalized, each bot version was frozen for use in competitions to ensure consistency. The iterative prompting process was characterized by a shift from foundational bot logic and implementation concerns to debugging and troubleshooting concerns when implemented into the pre-existing game environment. The development process naturally broke into three categories: foundational development and refactoring, debugging and troubleshooting, and dynamic testing integration. Table II summarizes the prompt distribution:

Within these percentages of conversations, we created a breakdown with a rough percentage of the total bot revisions that were related to a specific bot versus shared bot characteristics or behavior. Bot-specific challenges varied across

TABLE II
BOT CREATION PROMPT BREAKDOWN

Category	Prompt %	Example Prompt
Development	57%	Safe_select is being called on the CycleBot constantly instead of waiting
Debugging	29%	Getting this error: UnboundLocalError: cannot access local variable 'start'
Dynamic Testing	14%	I need the bots to be able to track metrics per game in order to perform the Monte Carlo Simulation

TABLE III
BOT-SPECIFIC REVISION BREAKDOWN

Bot Type	Focus Level	Primary Revision Topic
CycleBot	≈ 40%	Continuous selection and input waiting
PatternBot	≈ 20%	Integration into dynamic test
QLearningBot	≈ 15%	Reward mapping logic
RandomBot	≈ 10%	Integration testing
Shared Interfaces	≈ 15%	Bot interactions with each other

agents. Table III presents a summary of the development focus and key revision areas per bot with a Shared Interfaces row denoting focus on bots communicating with each other within the gaming environment.

The rest of this section details logical specifics and functional differences between each of the bots with a high level breakdown of how each bot determines its next move

1) *PatternBot*: PatternBot tries to spot patterns in opponent moves and capitalize on them. We instructed ChatGPT to have PatternBot track the latest moves of the opponent and look for simple patterns or frequencies. For instance, PatternBot would spot if the opponent is favoring a particular move or repeating a cycle. The bot then chooses the move that would beat the guessed next move of the opponent. If no apparent pattern is established, PatternBot may fall back to a default strategy (e.g., a random move or a static sequence). This heuristic approach, summarized in Figure 3, will try to take advantage of non-random patterns rapidly. For example, if the last three moves by the opponent were Rock, Paper, Rock, PatternBot may guess that "Rock" has high frequency and play Paper (which beats Rock) the next move. The next pseudocode illustrates a section of the PatternBot logic:

```
class PatternBot()
def choose_move(self, opponent_history):
    self.opponent_history = opponent_history
    last2 = opponent_history[-2:]
    if opponent_history[-1:i+2] == last2:
        return move_that_beats(predicted_next)
```

Fig. 3. PatternBot choose_move() Logic

2) *QLearningBot*: QLearningBot uses reinforcement learning by Q-learning to improve its strategy over time. The bot uses the repeated RPSLS game as an environment

to learn the best policy against the opponent. We instructed ChatGPT to use a simple Q-learning algorithm: the state can be represented by the opponent's last move (or last n moves) and the action is the bot's next move. At the completion of each round, the bot is rewarded (+1 for a win, -1 for a loss, 0 for a tie) and its Q-table updated as a result. With lots of rounds, the QLearningBot should learn what move will most probably lead to a win depending on what the last few moves from the opponent were. We allowed the bot to continue learning between matches (copying its Q-table over), essentially letting it learn as it plays against different opponents throughout the tournament. The code snippet in Figure 4 shows the core of QLearningBot's update rule:

```
def __init__(self, alpha=0.1, gamma=0.9, epsilon=0.1):
    self.last_state = None
    self.last_action = None
def choose_move(self, opponent_history):
    self.Q[state] = {m: 0.0 for m in RPSLS_moves}
    action = random_choice_of(RPSLS_moves)
    action = argmax(self.Q[state])
    self.last_action = action
    return action
def record_result(self, opponent_move, outcome):
    return
    self.Q[next_state] = {m: 0.0 for m in RPSLS_moves}
    self.Q[self.last_state][self.last_action] = new_value
```

Fig. 4. QLearningBot choose_move() Logic

3) *CycleBot*: CycleBot is deterministic and cycles through moves in a repeating manner. We chose a repeating cycle (Rock → Paper → Scissors → Lizard → Spock → repeat) and instructed ChatGPT to create a bot that remembers its last move and plays the subsequent one in the cycle. CycleBot demonstrates a predictable strategy by design. Figure 5 illustrates this strategy. This bot is an "exploitable" opponent to see how well PatternBot and QLearningBot perform, because if an opponent can recognize the cycle, they can always anticipate the next move and thwart it. CycleBot will not adapt strategy for any given opponent; it will just play its cycle regardless.

```
def __init__(self):
    self.moves = ['Rock', 'Paper', 'Scissors', 'Lizard', 'Spock']
    self.index = 0
def choose_move(self, opponent_history):
    move = self.moves[self.index]
    return move
```

Fig. 5. CycleBot choose_move() Logic

4) *RandomBot*: As its name implies, this bot makes completely random moves, every turn an independent uniform choice among the five options (shown in Figure 6). We instructed ChatGPT to make RandomBot play so that it uses a secure random generator (Python's random.choice, which is pseudorandom but adequate for a game). RandomBot is the game-theoretic best naive strategy, it cannot be exploited for a systematic error, nor can it exploit opponents' mistakes. Over several rounds, any non-random opponent will do marginally better than RandomBot (or break even, if both are random).

```
pythonCopyclass RandomBot(Bot):
    def choose_move(self, opponent_history):
        return random.choice(['Rock', 'Paper',
                              'Scissors', 'Lizard', 'Spock'])
```

Fig. 6. RandomBot choose_move() Logic

B. Game Environment Implementation and Validation

The game environment was built using PyGame and validated using Pytest and Hypothesis [15] to ensure both logical consistency and UI stability. Validation focused on two key properties:

- **Symmetry of Win Conditions:** For all actions a and b , if a beats b , then b must lose to a .
- **Environmental Stability:** The environment should remain robust and error-free over 1–100 rounds of randomly generated action pairs.

An example property-based assertion used to validate win condition symmetry is shown below:

```
assert(get_winner(a, b), get_winner(b, a)) in
[(0,0), (1,2), (2,1)]
```

All property tests and GUI automation scripts were executed against the final environment. During initial test runs, any failures—such as violations of the symmetry condition or crashes during randomized play—immediately triggered prompt-refine iterations. For example, when Hypothesis uncovered a counterexample to win-condition symmetry, we re-prompted ChatGPT, corrected the game logic, and re-ran the test. This process continued until no further failures occurred.

The finalized game engine passed 100% of Hypothesis property tests, confirming both logical correctness and resilience to randomized input sequences. Additionally, GUI automation tests—driven by Pytest and 'pygame.event.post()'—simulated full bot-vs-bot matches. These tests confirmed that the UI transitioned cleanly from selection to match mode and completed each game without crashes or hangs. The internal match-completion flag was set successfully in all cases.

These results demonstrate that the environment maintained both functional integrity and user-interface robustness under automated and manual interaction, forming a reliable foundation for bot performance evaluation.

C. Competitive Performance Evaluation

The results of the Monte-Carlo Simulation, comprising 4,000 matches per bot are presented in this section to assess the effectiveness of four AI agents, PatternBot, QLearningBot, CycleBot, and RandomBot, within a Rock-Paper-Scissors-Lizard-Spock (RPSLS) environment.

PatternBot achieved the highest win rate at 65.0%, demonstrating strong predictive capabilities and efficient adaptation based on opponent move sequences. It also exhibited one of the shortest average match durations (33.4 rounds).

QLearningBot followed with a 55.0% win rate, reflecting effective reinforcement learning but slightly lower performance consistency compared to PatternBot. Its matches were marginally longer on average (35.4 rounds), indicating a more exploratory decision-making process.

RandomBot showed moderate success with a 51.2% win rate, slightly above chance level, suggesting that unpredictability can yield reasonable results even in strategic settings. However, its longer average match length (38.1 rounds) reflects inefficiency in quickly resolving matches.

CycleBot underperformed significantly with a 16.2% win rate and 1,000 timeouts, underscoring the predictability and exploitability of its deterministic move sequence. It had the shortest average rounds to completion (29.6), though likely due to being quickly defeated by more adaptive bots.

TABLE IV
OVERALL SUMMARY OF BOT COMPETITION

	PatternBot	Q-LearningBot	CycleBot	RandomBot
Matched Played	4000	4000	4000	4000
Wins	2602	2199	649	2050
Losses	1398	1801	2351	1950
Win Percentage	65.0%	55.0%	16.2%	51.2%
Timeouts	0	0	1000	0
Avg. Rounds to Finish	33.4	35.4	29.6	38.1

The results, compiled in Table IV, highlight that pattern recognition and reinforcement learning outperform fixed or random strategies in adversarial, adaptive environments. Cycle-based approaches are highly vulnerable due to their lack of adaptability, while randomness provides limited competitive advantage absent strategic reasoning.

D. Security Assessment of AI-Generated Code

The multi-tool static analysis revealed a relatively secure, though not flawless, implementation of the RPSLS simulation environment. No high- or medium-severity vulnerabilities were detected in any Python file, suggesting that the source code avoids exploitable flaws. However, these tools consistently reported low-severity vulnerabilities that point to software engineering practices misaligned with secure coding guidelines.

1) *Findings from Bandit and GitLab SAST:* Static analysis revealed no high- or medium-severity flaws in the RPSLS environment, but repeated low-severity issues exposed meaningful trends in AI-generated code. Bandit and GitLab SAST flagged recurring instances of CWE-338 [16] (use of the insecure random module) and CWE-703 [17] (use of assert for logic enforcement) shown in Figure 7. While these patterns did not affect functionality, they highlight how AI systems adopt insecure defaults. CWE-338 suggests the model drew from the overwhelming prevalence of random in training data, overlooking security-conscious alternatives like secrets. CWE-703 reflects a bias toward readability and brevity, where assert is often used in community code despite its removal in optimized execution. In contrast, human-written portions relied more on explicit validation and defensive handling,

underscoring the gap between secure engineering intent and AI's surface-level mimicry.

```
Code scanned:
  Total lines of code: 649
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 22
    Medium: 0
    High: 0
  Total issues (by confidence):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 22
Files skipped (0):
```

Fig. 7. Results after running Bandit against the source code

2) *Findings Identified by ChatGPT:* Beyond these core vulnerabilities, ChatGPT's own review raised concerns with unbounded simulations, weak GUI input validation, and race conditions in automated testing. Though not critical in this academic setting, they demonstrate the tendency of AI-generated code to favor simplicity over resilience. Together, the findings show that even when secure in practice, AI-authored code can inherit systemic weaknesses from its training data. Addressing these issues—by adopting secure randomness, replacing asserts with explicit error checks, and enforcing stronger input and concurrency controls—would strengthen the environment and prevent such patterns from propagating into higher-stakes deployments.

These findings underscore the importance of secure software engineering practices, even in academic or simulation settings. While the application does not handle real-world sensitive data, its design decisions reflect patterns that could translate into risk if adapted for broader deployment, such as in online educational platforms or AI teaching tools. Addressing these issues would involve targeted refactoring, improved input validation, controlled logging, and enforcing thread safety through locks or event synchronization. The cumulative outcome of the security evaluation is a call for preventative hardening, not in response to active threats, but to future-proof the system and reinforce its trustworthiness under scale or real-world use.

V. LIMITATIONS

Despite the promising outcomes, our study has several important limitations:

A. Scope and Generalizability

We worked with a relatively simple two-player game environment and specific agent strategies. The results and experiences might not directly generalize to more complex domains or larger codebases. In a real-world cybersecurity simulation or a more complex game (with many moving parts, network communication, etc.), the effectiveness of ChatGPT in generating correct and secure code remains unproven

B. AI Model Limitations and Biases

ChatGPT's knowledge and outputs are based on its training data, which might not perfectly cover our game's niche or could include biases. We observed that ChatGPT sometimes made assumptions or used coding patterns that, while generally sound, were not optimized for our specific game. For instance, it initially prioritized code readability and clarity over efficiency. In a larger-scale scenario, such choices could impact performance. Moreover, if the model had any inherent biases (for example, favoring a certain style of play or coding), our agents might inadvertently reflect those. We did not see any harmful biases in this context, but the risk exists in general.

C. Complexity and Context Window

Generating even our moderately sized agents (hundreds of lines of code) pushed the limits of what we could comfortably do in one prompt with GPT-4. For larger programs, one would need to break down the task. ChatGPT has a limited context window, so developing a complex multi-file project would require careful orchestration. Our approach did not tackle this complexity; we dealt with single-file bot logic and a relatively contained environment. Thus, scalability of this approach is a concern: managing coherence and correctness across a big project with only conversational prompts would be challenging.

D. Evaluation Metrics

Our evaluation focused primarily on win/loss outcomes and basic gameplay statistics. We did not deeply assess other aspects such as the explainability of the AI agents' decisions or their robustness to changes in opponent strategy. These could be seen as limitations in evaluating the AI agents' quality. For example, a strategy might win often but be based on brittle assumptions that fail if the environment is slightly altered – we did not test the agents against a wide variety of opponents or variations of rules.

Each of these limitations points to areas where caution is needed in interpreting our results, as well as opportunities for further research to extend and deepen the analysis.

VI. CONCLUSION & FUTURE WORK

This study evaluated OpenAI's ChatGPT-4o as an autonomous developer of competitive cyber-gaming agents by iteratively generating, analyzing, and testing code for a weighted RPSLS game. The LLM produced functional bots that outperformed random and fixed strategies, but required repeated prompt refinements and exhibited low-severity security issues (e.g., unsafe randomness, assert-based checks). Targeted refactoring—hardening randomness, sanitizing inputs, controlling logs, and enforcing thread safety—can mitigate these risks. Overall, generative AI enhances but does not yet replace human developers for secure, production-grade code. Future work includes adaptive prompting, embedded security constraints, formal verification, and improved human–AI collaboration.

REFERENCES

- [1] A. A. Mahyari, "Harnessing the power of llms in source code vulnerability detection," in *MILCOM 2024-2024 IEEE Military Communications Conference (MILCOM)*. IEEE, 2024, pp. 251–256.
- [2] S. Martínez-Fernández, J. Bogner, X. Franch, M. Oriol, J. Siebert, A. Trendowicz, A. M. Vollmer, and S. Wagner, "Software engineering for ai-based systems: a survey," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 2, pp. 1–59, 2022.
- [3] M. Alenezi and M. Akour, "Ai-driven innovations in software engineering: a review of current practices and future directions," *Applied Sciences*, vol. 15, no. 3, p. 1344, 2025.
- [4] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [5] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv e-prints*, pp. arXiv–2407, 2024.
- [6] Defense Advanced Research Projects Agency, "Cgc: Cyber grand challenge," accessed: May 2, 2025. [Online]. Available: <https://www.darpa.mil/research/programs/cyber-grand-challenge>
- [7] The Art of Machinery, "Glico (weighted rock paper scissors)," 2020. [Online]. Available: https://theartofmachinery.com/2020/05/21/glico_weighted_rock_paper_scissors.html
- [8] GitLab Inc., "Static application security testing (sast)." [Online]. Available: <https://about.gitlab.com/stages-devops-lifecycle/static-application-security-testing/>
- [9] PyCQA, "Bandit: A security linter for python (version 1.7.4)." [Online]. Available: <https://bandit.readthedocs.io/>
- [10] M. Kholoosi, M. and C. T. C. for Research on Engineering Software Technologies, "A qualitative study on using chatgpt for software security: Perception vs. practicality," 2024. [Online]. Available: <https://arxiv.org/abs/2408.00435v1>
- [11] M. I. Hossain and R. Hasan, "Enhancing security in ambient intelligence: A stride threat modeling perspective," in *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. IEEE, 2023, pp. 1–6.
- [12] R. Raghi, K. K. Sudha, M. S. A. and S. J. S., "Software development automation using generative ai," in *2024 International Conference on Emerging Research in Computational Science (ICERCS)*, Coimbatore, India, 2024, pp. 1–6.
- [13] Tony *et al.*, "Prompting techniques for secure code generation: A systematic investigation," 2024, preprint, arXiv:2407.07064v2. [Online]. Available: <https://arxiv.org/abs/2407.07064v2>
- [14] P. Shinnars, "pygame." [Online]. Available: <https://www.pygame.org/>
- [15] R. MacIver, D. and Z. Hatfield-Dodds, "Hypothesis," Python library, PyPI. [Online]. Available: <https://pypi.org/project/hypothesis/>
- [16] MITRE, "Cwe-338: Use of cryptographically weak pseudo-random number generator (prng)." [Online]. Available: <https://cwe.mitre.org/data/definitions/338.html>
- [17] —, "Cwe-703: Improper check or handling of exceptional conditions." [Online]. Available: <https://cwe.mitre.org/data/definitions/703.html>